



Odhadovač hlasitosti na Raspberry Pi

Ročníková práce z předmětu PSS

Vít Jandoš

C3c

Zadání práce

Připojit senzor k RPI, v pravidelné intervalu zaznamenávat hodnoty do DB a následně nějak graficky znázorňovat na webu

Obsah

Zadání práce	2
Úvod	4
Rozbor	5
Hardware	5
Software	5
Realizace	7
Příprava a zapojení	7
Nahrávání	8
Odhad hlasitosti	8
Vytváření venv	8
Odhad	8
Zapisování hodnot	9
Databáze	9
Komunikace s databází	10
Grafy na webu	10
Live hlasitost	11
Závěr	13
Seznam obrázků	14

Úvod

Cílem projektu bylo připojit mikrofon na Raspberry Pi, který bude pravidelně zaznamenávat zvuk a odhadovat něj aktuální hlasitost, tyto hodnoty následně ukládat do databáze.

Z těchto dat potom vykreslovat grafy na webu s možností vybrat různé časové období. Kromě grafů jde na webu taky najít pár zajímavostí jako třeba nejhlasitější hodina ve dne. Nakonec jsem taky přidal zobrazování live hlasitosti.

Mít tento web přístupný odkudkoliv se hodí, protože jde vidět, jestli míra hlasitosti nepřesahuje doporučenou hranici. Taky jde například vidět kdy se vzbudil ráno první člověk, jestli je někdo doma a nebo kdy se vařilo v kuchyni.

Rozbor

Hardware

Jako mikropočítač jsem zvolil **Raspberry Pi Zero 2 WH**, protože jde o jeden z nejlevnějších modelů, který zároveň splňuje požadavky projektu. Zařízení podporuje operační systém **Linux**, disponuje GPIO piny pro připojení externích komponent a nabízí dostatečný výkon i operační paměť pro plynulou komunikaci s databází.

Pro snímání zvuku jsem zvolil digitální mikrofon **ICS-43434 I2S MEMS Microphone**. Jedná se o cenově dostupný mikrofon schopný zaznamenávat zvuk ve vysoké kvalitě s tolerancí přibližně ± 1 dB. Díky tomu je vhodný pro domácí nahrávání a orientační měření hlasitosti.

Jako paměťové médium mikropočítače slouží **32GB microSD karta třídy A2**, která obsahuje operační systém i data aplikace. Vzhledem k tomu, že zařízení běží nepřetržitě a často zapisuje data, byla zvolena karta s dostatečným výkonem pro časté zápisy.

Napájení je zajištěno pomocí standardního napájecího adaptéru a **micro-USB kabelu**, který je kompatibilní s napájecím konektorem Raspberry Pi. Spotřeba zařízení je relativně nízká, proto postačuje běžný napájecí adaptér.

Na zkalkulování mikrofону jsem využil hlukoměr **JOY-IT JT-SLM01**, protože je to jediný dostupný hlukoměr na alze.

Software

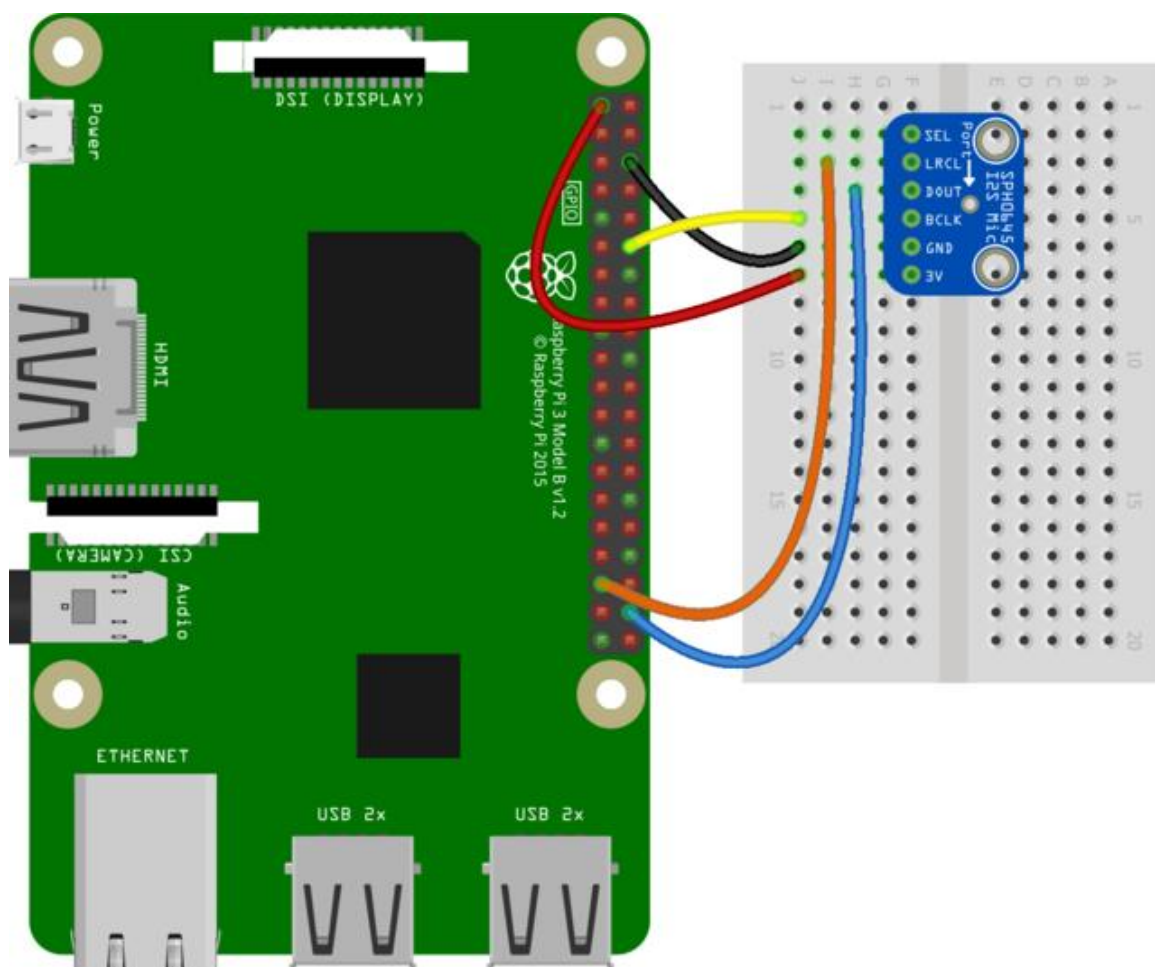
Jako operační systém jsem použil **Debian GNU/Linux 13**, konkrétně verzi verze **Trixie**. Tento systém je pro zařízení Raspberry Pi stabilní, dobře podporovaný a vhodný pro dlouhodobý provoz.

Hlavním programovacím jazykem byl **Python**, protože nabízí velké množství knihoven pro práci se zvukem, hardwarem i databázemi. Další výhodou je jednoduchá syntaxe, která programování zjednodušuje a usnadňuje pochopení jazyka .

Pro ukládání dat jsem zvolil databázi **Supabase**. Jedná se o službu typu **Backend as a Service**, která umožňuje komunikaci aplikace s databází bez nutnosti vytvářet vlastní backend server. Také nabízí funkci live channel, kterou využívám pro živé zobrazování aktuální hlasitosti na webu.

Frontend webové aplikace byl vytvořen pomocí jazyků **HTML**, **CSS** a **JavaScript**. Pro vykreslování grafů na elementu `<canvas>` byla použita knihovna **Chart.js**, která umožňuje jednoduše zobrazovat časové průběhy naměřených hodnot.

Pro hosting webové části projektu byla použita služba **GitHub Pages**, která umožňuje zdarma publikovat statické webové stránky přímo z repozitáře na platformě **GitHub**.



Obrázek 1 - Blokové schéma zapojení

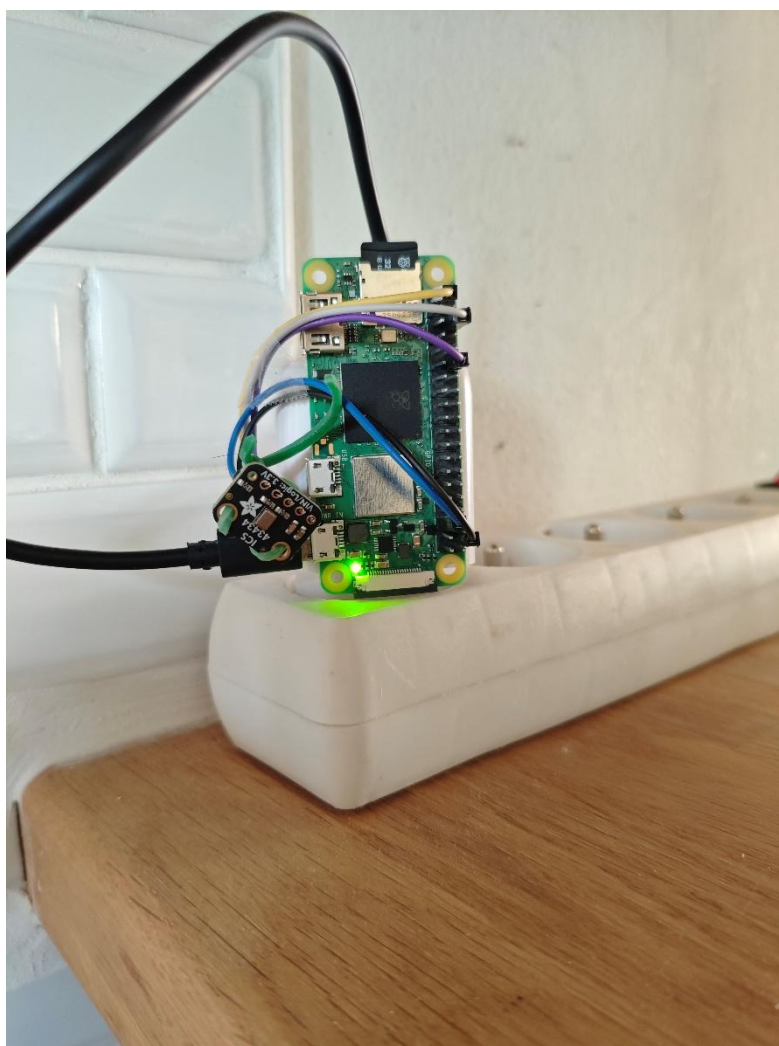
fritzing

Realizace

Příprava a zapojení

Nejprve jsem musel nakoupit všechny potřebný hardware. Když jsem ho měl tak jsem nejprve nainstaloval operační systém pomocí Raspberry Pi Imager. Tento program instalaci usnadnil a umožnil jednoduché nastavení ssh a připojení k wifi. Nejdříve jsem Pi poprvé spustil, připojil jsem se na něj přes ssh a otestoval jestli funguje. Poté jsem ho opět vypnul a zapojil mikrofon na tyto piny:

- **3V** – Napájení, pin číslo 1
- **GND** – Uzemění, pin číslo 6
- **BCLK** – Clock, pin číslo 12
- **DOUT** – Data z mikrofonu, pin číslo 38
- **LRCLK** – Výběr kanálu kam má posílat data, pin číslo 35
- **SEL** – Který kanál je mikrofon, odpojeno, protože mám jen jeden



Obrázek 2 - Fyzické zapojení

Nahrávání

Se zapojeným mikrofonom jsem musel nejprve nakonfigurovat ALSA (Advanced Linux Sound Architecture). To jsem udělal editem souboru **/boot/firmware/config.txt**, kde jsem musel odkomentovat nebo dopsat tyto řádky:

```
# Uncomment some or all of these to enable the optional hardware interfaces
#dtparam=i2c_arm=on
dtparam=i2s=on
#dtparam=spi=on

# Enable audio (loads snd_bcm2835)
dtparam=audio=on

# Additional overlays and parameters are documented
# /boot/firmware/overlays/README
dtoverlay=googlevoicehat-soundcard
```

Obrázek 3 - Konfigurační soubor pro firmware

Poté jsem soubor uložil a rebootoval Raspberry Pi. Po načtení už systém viděl můj mikrofón, to se dá ověřit pomocí příkazu `arecord -l`. V tu chvíli jsem mohl začít nahrávat zvuk.

Odhad hlasitosti

Vytváření venv

Pro odhad hlasitosti jsem vytvořil skript v jazyce **Python**. Pro to, aby fungoval bylo nutné využít externí knihovny, jako například **sounddevice**, které umožňují práci se zvukovým vstupem. Ty ale nejsou povolené instalovat kvůli stabilitě.

Z tohoto důvodu jsem si vytvořil virtuální prostředí (**venv**), ve kterém běží izolovaná instance Pythonu. Do tohoto prostředí je možné instalovat potřebné knihovny bez ovlivnění systémové instalace.

Odhad

Na odhad jsem použil tento skript, který pomocí **sounddevice** nahraje vzorek dlouhý půl sekundy, následně ho převede do rozmezí -1 až 1. Poté jsou všechny hodnoty převedeny na absolutní hodnotu, protože při výpočtu hlasitosti je důležitá jen velikost síly signálu.

Ze vzniklého pole potom vezme 5% nejsilnějších signálů, aby nezanikly mezi ostatními slabými signály, kterých je mnohem víc.

Z těchto vybraných hodnot je následně vypočítán průměr. Tento průměr je poté převeden na relativní decibely (dBFS) a upraven pomocí referenčních hodnot, které jsem nastavil později podle skutečného hlukoměru.

Skript jsem nastavil v **cronu**, aby se spouštěl každou minutu.


```
#!/home/vitek/funkcniPython/bin/python3
import sounddevice as sd
import numpy as np
import datetime
import os
import psycopg2

DURATION = 0.5
SAMPLE_RATE = 48000
CHANNELS = 1
REFERENCE_SPL = 94
REFERENCE_DBFS = 10

sd.default.device = 'snd_rpi_googlevoicehat_soundcar'

audio = sd.rec(
    int(DURATION * SAMPLE_RATE),
    samplerate=SAMPLE_RATE,
    channels=CHANNELS,
    dtype='int32'
)

sd.wait()

audio_float = audio.flatten() / (2**31)
abs_audio = np.abs(audio_float)
loudest_samples = np.sort(abs_audio)[: -1][:int(len(abs_audio) * 0.05)]
avg = np.mean(loudest_samples)

if avg == 0:
    print("Žádný signál.")
    exit()

dbfs = 20 * np.log10(avg)
hlasitost = round(dbfs - REFERENCE_DBFS + REFERENCE_SPL)
```

Obrázek 4 - Část scriptu na odhad hlasitosti

Zapisování hodnot

Jelikož jsem zatím neměl vytvořenou databázi tak jsem hodnoty zaznamenával do CSV souboru, který jsem měl sdílený s počítačem pomocí programu samba.

Databáze

Jako svojí databázi jsem zvolil **Supabase**. Je zdarma a funguje jako Backend as a service. To znamená, že dokáže komunikovat s frontendem přímo, tím že poskytuje javascriptu knihovnu klienta přes objekt klienta. To výrazně zjednodušuje vývoj aplikace a snižuje množství potřebného kódu.

V databázi jsem vytvořil jen jednu tabulku, ve které se ukládají jednotlivé záznamy času a hodnoty hlasitosti.

Nakonec jsem vytvořil také několik pohledů (**views**), které slouží k výběru dat podle jejich stáří. Další pohledy jsou zaměřeny na zjištění různých zajímavostí, například nejhlasitější hodiny během dne nebo maximální naměřené hodnoty.

```
CREATE VIEW view_den AS
SELECT hlasitost, datum_cas FROM zaznam
WHERE datum_cas >= NOW() - INTERVAL '1 day'
ORDER BY datum_cas;

CREATE VIEW view_dvanact AS
SELECT hlasitost, datum_cas FROM zaznam
WHERE datum_cas >= NOW() - INTERVAL '12 hour'
ORDER BY datum_cas;

CREATE VIEW view_prumer AS
SELECT round(avg(hlasitost)) as prumer FROM zaznam;
```

Obrázek 5 - Příklad pohledů z databáze

Komunikace s databází

Raspberry pi zaznamenává hodnoty a ukládá je do databáze, do které se připojuje přímo přes knihovnu **psycopg2**. Svůj connection string mám uložený jako environmental variable, aby ve scriptu nebylo vidět moje heslo.

V javascriptu mám udělaného supabase **klienta**, od kterého získávám data do grafů z už zmíněných selectů.

```
const database = supabase.createClient('https://tgyjtmpoyobshnjzukurz.supabase.co', 'sb_publishable_5aZ7f4gDqje27zLDCwCGKw_xMEEp0gi');

async function vykreslGraf(typGrafu, pocetTicku) {
  const { data, error } = await database
    .from(typGrafu)
    .select('*')

  const hodnoty = data.map(d => d.hlasitost)
```

Obrázek 6 - Použití klienta Supabase

Grafy na webu

K vykreslování grafů používám knihovnu **Chart.js**, která vykresluje grafy na **<canvas>**. Vybral jsem si jí, protože je jednoduchá na pochopení a dává mi velké množství možností na úpravu grafů.

Pro všechny grafy mám stejnou metodu, protože zobrazují stejná data, jen jiné množství. Jediné co se mění jsou labely, kde čím delší období to je, tím méně detailní datum a čas je vidět.

```

const hh = String(datum.getHours())
const mm = String(datum.getMinutes()).padStart(2, '0')
const dd = String(datum.getDate());
const mes = String(datum.getMonth() + 1).padStart(2, '0');

if(typGrafu=="view_den" || typGrafu=="view_dvanact"){
return `${hh}:${mm}`
}
if(typGrafu=="view_tri_dny"){
return `${dd}/${mes} ${hh}:${mm}`;
}
if(typGrafu=="view_tyden"){

const aktualniDatum = `${dd}/${mes}`
if (aktualniDatum == predchozi.datum) {
return ''
}

predchozi.datum = aktualniDatum
return aktualniDatum
}
return "zadneDatum"
}

```

Obrázek 7 - Metoda na nastavování formátu data

Live hlasitost

Nakonec jsem se rozhodl ještě přidat funkci zobrazování live hlasitosti. Pro to byla potřeba posílat data na web bez ukládání do databáze. Využil jsem služby co poskytuje Supabase jménem **live channel**.

Základ na Raspberry Pi byl stejný script na měření, jen je místo ukládání posílá na live channel.

Narozdíl od ukládacího scriptu nevyužívá cron na pravidelné spuštění, ale běží celou dobu a jen dvě sekundy čeká před dalším odesláním dat.

Na to, aby běžel neustále a nespádl je vytvořený jako **systemd service**. To se udělá přidáním souboru do **/etc/systemd/system** se jménem xxxxxx.service. Tam se definuje, co má služba

dělat a jaké má vlastnosti. Já tam například mám ať se restartuje když se zasekne.

```
[Unit]
Description=Broadcastuje hlasitost na live channel supabase
After=network.target

[Service]
ExecStart=/home/vitek/funkcniPython/bin/python3 /home/vitek/broadcast
Restart=always
RestartSec=10
WatchdogSec=60
User=vitek
StandardOutput=journal
StandardInput=journal

[Install]
WantedBy=multi-user.target
```

Obrázek 8 - Systemd služba

Závěr

Cílem tohoto projektu bylo vyrobit funkční systém pro měření hlasitosti pomocí digitálního mikrofonu, následné ukládání a vizualizaci těchto dat. To jsem do jisté míry splnil. Jediné co by se dalo zlepšit je přesnost měření. Problém je, že mikrofon, co jsem použil je citlivý pouze na nízkou vzdálenost, takže nemusí registrovat zvuky z druhé strany pokoje. Tomuto by se dalo předejít využitím zařízení, které je dělané na měření hlasitosti. To ale nebyl cíl projektu.

V průběhu se jsem se naučil pracovat se Supabase, Chart.js a Pythonem. Taky jsem si zopakoval Linux. Díky tomuto projektu jsem udělal první webovou aplikaci, kde jsem všechno dělal sám včetně backendu, se kterým jsem se lépe seznámil.

Během vytváření projektu se samozřejmě vyskytly nějaké problémy. Největší z nich byl nefungující nahrávání zvuku, kde mikrofon byl správně zapojený, ale neposílal žádný signál. Nakonec jsem přišel na to, že se piny špatně dotýkaly vodičů mikrofonu. Spravil jsem to gumičkou.

Aplikace nepotřebuje žádnou údržbu. Pokud selže live vysílání hlasitosti tak se to po nějaké době samo restartuje. Pokud z nějakého důvodu selže ukládání do databáze tak se maximálně vynechá jeden záznam, což moc nevadí, protože se nahrávají každou minutu.

Seznam obrázků

Obrázek 1 - Blokové schéma zapojení.....	6
Obrázek 2 - Fyzické zapojení	7
Obrázek 3 - Configurační soubor pro firmware	8
Obrázek 4 - Část scriptu na odhad hlasitosti.....	9
Obrázek 5 - Příklad pohledů z databáze	10
Obrázek 6 - Použití klienta Supabase	10
Obrázek 7 - Metoda na nastavování formátu data.....	11
Obrázek 8 - Systemd služba	12